

Aufgabenplan er MASTER TODO- Mobile Native App



09.07.2026
Modul 335-Mobile
Apps realisieren
Elias Kaiser
Christopher Biebow

Inhaltsverzeichnis

User Story 1..... 3

User Stories Aufgabenplaner Master Todo Native App 3

User Story 2 3

User Story 3 4

User Story 45

User Story 5.....5

UI-Skizzen mit Visily7

.....7

Datenbankmodell Aufgabenplaner Elias 8

TaskItem Entity : 8

User Stories Aufgabenplaner Master Todo Native App

User Story 1

Alle Tasks übersichtlich anzeigen in Liste

- 1) Überblick beim Starten: Als orientierungsloser User möchte ich alle meine Tasks übersichtlich angezeigt bekommen um meine

Akzeptanzkriterien:

- Liste mit Tasks aus ListView
- Taskadapter, um zwischen ListView mit Taskactivity zu verbinden und zwischen Formular und Start-Activity zu wechseln
- MainActivity ist mit Manifest-File richtig konfiguriert damit die

User-Story: Tasks in List View übersichtlich anzeigen

Testvoraussetzung: MainActivity mit Taskadapter und Activitys zu verbinden ist vorhanden sowie gesamte Mobile-App lässt sich im Emulator erfolgreich starten

- Leere Liste wird angezeigt auf Emulator
- Neue Aufgaben werden strukturiert innerhalb der ListView angezeigt und gerendert
- Tasks können nach Attributen wie Beschreibung, Taskname und Schwierigkeit gefiltert werden und mit dem Reset-Button wieder zurückgesetzt werden
- Ein TaskAdapter zur Verbindung der Datenquelle mit der ListView ist implementiert
- Gespeicherte Aufgaben werden angezeigt in ListView nachdem Task hinzugefügt wurde
- Fehlermeldung/Alert-Meldung mit Input-Validation falls ein Pflichtfeld fehlt

User Story 2

Neue Task erfassen können

- 2) Als gestresster User möchte ich einen neuen Task erfassen können und zur Liste hinzufügen können um meine Todo's im Überblick behalten können

Akzeptanzkriterien:

- Formular öffnet sich nachdem Button + "Task hinzufügen" geklickt wurde
- Titel, Beschreibung und Schwierigkeit der Task ist Pflichtfeld und wird sofort in der Liste angezeigt
- Damit neue Tasks hinzugefügt werden benötigt es zwingend der Name der Task(name) und die Beschreibung als Attribute um neue Tasks hinzuzufügen
- Resetbutton existiert, um gefilterte Daten zurückzusetzen
- Leerer Titel oder Beschreibung wird nicht akzeptiert und benötigt zwingend ein String
- Im Titel kann beliebiger Text eingegeben werden
- Ein neuer Task kann jederzeit via „Abbrechen“ das Fenster abgebrochen werden ohne dass die Inputdaten gespeichert werden

Testvoraussetzung:

- MainActivity und TaskActivity verbinden sich via Taskadapter um zwischen Formulare zu wechseln
- Button um Task hinzufügen «+Add task» und «Abbrechen» wurde in XML und Activity implementiert
- MainActivity ist sichtbar und startet erfolgreich ohne Fehler

User-Story:

Neue Tasks zur Liste hinzufügen

User Story 3

Task aus erledigen und in Taskliste abhaken

- 3) Als beschäftigter User möchte ich die erledigten Tasks abhaken oder ganz aus der Liste löschen können, damit meine Liste immer aktuell bleibt und ich meinen stetigen Fortschritt zu sehen

Testvoraussetzung:

- Alle Listenelemente besitzen Checkboxes, um erledigte Tasks durch Click-Listener zu speichern
- Methode getView wurde in MainActivity und in TaskListActivity implementiert um Task zu erledigen mit verstecktem «Erledigt» Button
- Mobile App startet erfolgreich in VDI im Emulator bzw in der MainActivity und zeigt Taskformular an
- Interface OnTaskClickListener mit Methoden onTaskCheckChanged und onTaskDeleted existiert
- Innerhalb des TaskAdapter wurde eine Checkbox implementiert mit einem onCheckedChangeListener und einem updateStrokeThrough-Methode welche das UI sofort aktualisiert und durchstreicht

Akzeptanzkriterien:

- Nachdem Markieren einer neuen Task nach dem erledigen wird sie durchgeschtrichen und die UI der ListView
- Daten weden lokal gespeichert in DAO
- Nach Neustart der VDI im Emulator bleibt Task durchgestrichen
- Wenn ein User auf die Task cklickt wird die Task sofort durchgestrichen und durch erneutes Klicken in den Zustand «offen» zurückgesetzt
- Versteckter “Button” mit getView Methode in MainActivity und TaskActivity vorhanden
- Daten aus TaskListActivity werden asynchron über das Room-DAO-Objekt in der lokalen Datenbank abgespeichert, damit die Tasks nach dem Neustart des Emulators persitent gespeichert bleiben und durchgestrichen bleiben
- TaskAdapter enthält Hilfsmethode Striketrough-Effekt und Click-Listener für die Checkbox sobald eine Task durchgestrichen wird,das UI automatisch updated mit der updateStrokeThrough

User Story 4

Neue Task aus Liste löschen

- 1) Als User möchte ich gespeicherte Tasks aus der Liste jederzeit entfernen können um den aktuellen Zustand der Taskliste zu erhalten

Akzeptanzkriterien:

- Beim Klick auf den Löschen-Button (oder Long-Press) wird der Task ohne Umwege aus der ListView entfernt
- TaskAdapter triggert eine Aktion um Liste nach Neustart sofort aktuell ist mit den neusten Daten
- Try/Catch-Block mit Alert-Meldung Execeptionhandlugn wenn mehr als 10 Tasks hinzugefügt wurden erscheint eine Fehlermeldung

Testvoraussetzung:

- MainActivity mit Emulator lässt sich erfolgreich starten mit VDI
- TaskItem wurde erfolgreich mit Taskadapter verbunden
- Gelöschte Tasks werden dauerhaft aus der DAO-Objkt bzw. der lokalen Room-Datenbank entfernt und erscheinen nach dem Neustart der App nicht wieder
- Delete Button «Löschen» existiert in Taskliste um einzelne Task zu löschen aus Liste sowie Abbrechen um Daten nicht zu speichern in Liste
- Müllerimer-Icon aus dem UI-Entwurf mit SetOnClickListener neben Task UI existiert und greift Methode onTaskDeleted auf um die Task aus der lokalen Room-DB zu entfernen

User Story 5

Task persistent auf Liste in Room-Datenbank speichern

- 4) Als gestresster User möchte ich, dass meine Tasks nach dem Neustart in der Liste persistent gespeichert werden in der Datenbank damit ich nicht jedes mal die Task manuell hinzufügen muss

Akzeptanzkriterien

- -Daten werden lokal gespeichert
- Jedes Listenelement besitzt eine eindeutige Lösch-Button

Wenn die App komplett geschlossen und neu gestartet wird (oder die VDI im Emulator neu startet), wird der exakte Zustand (durchgestrichen oder offen) aller nicht-gelöschten Tasks korrekt aus der Datenbank geladen und visualisiert

- Alle Attribute Beschreibung, Priorität, Menge Taskname und Schwierigkeit des DAO stimmen überein mit Task-Objekt überein für die Persistente Speicherung
- Nach App-Neustart sind alle Aufgaben sichtbar

Testvoraussetzung:

- RoomDatabase wurde erstellt mit DAO-Objekt mit Datenbankstruktur wurde aufgesetzt und AppDatabase
- MainActivity lässt sich erfolgreich starten im Emulator-Modus der VDI und Task werden angezeigt sowie Formular wechseln funktioniert
- Es existiert eine Liste mit mindestens 3-Beispieltest-Tasks in der ListView i, das Verhalten beim Scrollen und Interagieren zu prüfen
- Android Studio läuft erfolgreich mit VDI im Emulator Modus in der MainActivity und wird stetig im Logcat überwacht
- - Datenbankoperationen mit Queries sind funktionsfähig um nur bestimmte Tasks abzugeben, COUNT, SELECT ALL, DESC,DELETE)
- MainActivity lässt sich im Emulator starten und AddTaskActivity wurde erstellt mit zugehörigem task_add.xml Files um neuen Task hinzufügen

-Nach dem Beenden der App (Kill-Prozess)der App im Emulator Neustart Button wird die App mit den Daten neu gerendert und verschwinden nicht

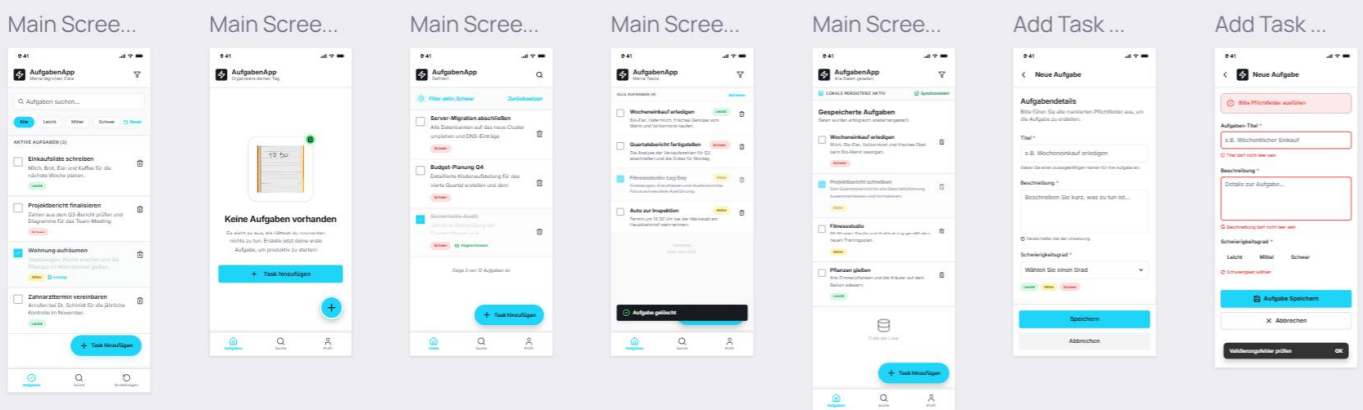
-Statusänderungen (Erledigt) oder Löschungen (aus Story 3) werden synchron in der Datenbank aktualisiert, sodass auch diese Änderungen nach einem Neustart bestehen bleiben

Userstory: NeuTask persistent in Liste speichern in lokalen SQL-Lite-DB

UI-Skizzen mit Visily

- MainActivity
- Converters
- TaskListActivity
- Taskadapter
- Task-Entity
- FilterActivity
- Task mit Konstruktor und Attributen der Aufgabe für Liste
- TaskAdapter
- TaskItem
- TaskItemDao

3 Activitys benötigt: MainActivity, AddTaskItem um zwischen Home-Screen wo alle Task bzw die Liste angezwit wird und zwische Input-Formular der Task welche hinzugefügt wechseln via TaskAdapter zu switchen



-Link zu VisilyBoard der UI-Screens der Aufgabenplaner -Mobile -App
<https://app.visily.ai/projects/231779b9-ca57-4935-a952-12019f7d8788/boards/2659023>

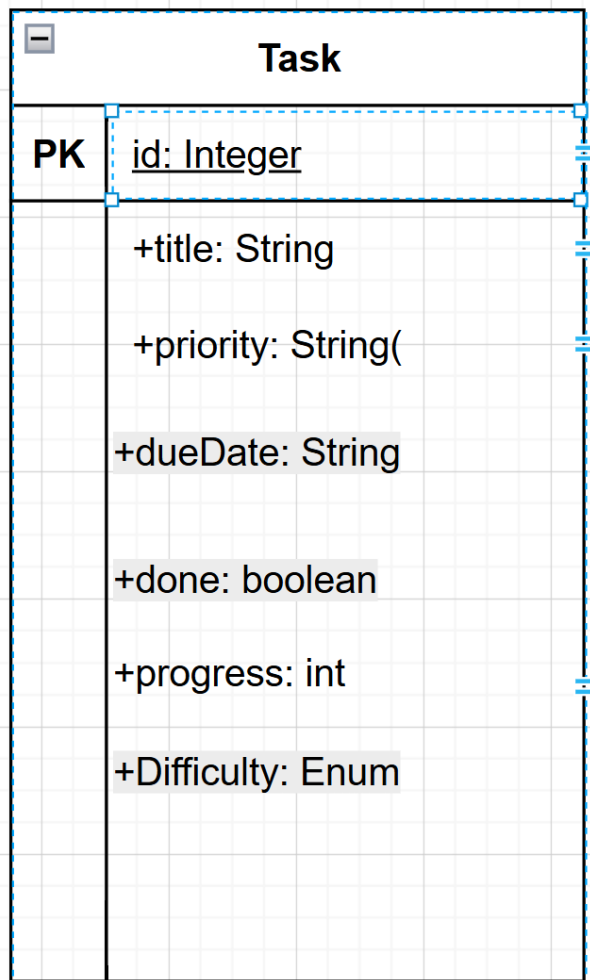
Datenbankmodell Aufgabenplaner Elias

TaskItem Entity :

Feld	Typ	Beschreibung
id	Int	Autogenerierter Primärschlüssel für eindeutige Identifikation der Task anhand der ID , damit keine doppelten Einträge entstehen
title	String	Titel ist Pflichtfeld für die Task
description	String	Beschreibung der Task ist Pflichtfeld
priority	String	Priorität oder mit Standardwert beispielsweise Mittel,LEICHT und SCHWER Pflichtfeld Textfeld für Wichtigkeit der Task
dueDate	String	Timestamp wann Task erledigt wurde mit Kalenderformat (Jahr,Monat,Tag,Minute)
done	boolean	Erledigt Status mit Checkbox und Task

		durchgestrichen wird und auf true gesetzt ist (Ja/Nein) für Textfeld
difficulty	Enum	Status mit Enum entweder LEICHT,MITTEL oder SCHWER gesetzt um die Schwierigkeit mit eibnem Dropwdown-Menu darzustellen
progress	int	Prozentzahl um in Taskbar Anzahl erledigter Task der gesamt Taskliste anzuzeigen(bsp: 40 %) und der Progress visuell durch eine grüner Balken darzustellen

Datenbankmodell Task Entity für Dao-Objekt:



Projektversionierung mit Github

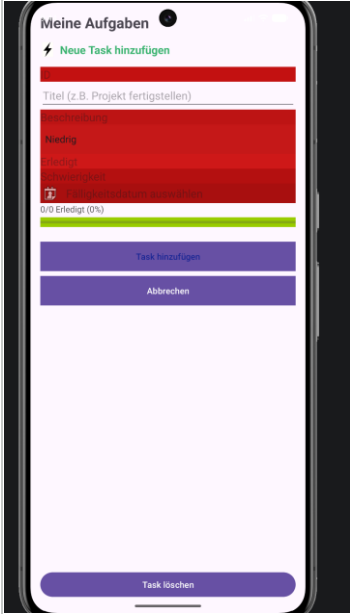
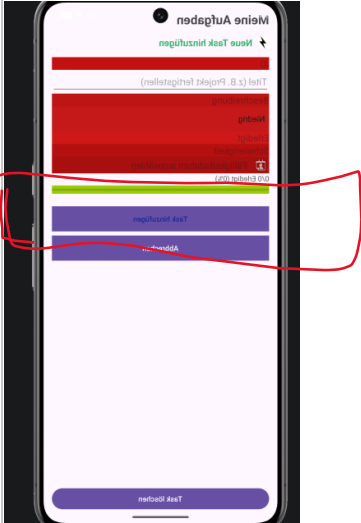
1. Neues Githubrepository initialisieren → `git init`
2. Gitprojekt per Remoteserver zugreifen und Origin URL setzen → `git remote set-url origin https://github.com/elias4864/Aufgabenplanner-Mobile-App`
3. Neuer Branch feature/task hinzufügen erstellen → `git checkout -b feature/task`
4. Lokale Änderung commiten und auf Remote BRnach /feature/task pushen auf Remoteserver
5. Für Alle Files für die Task und Datenbankpersistenz jeweils Code nach Thema gliedern trennen und auf eigenem Branch versionieren (Feature-Branching-Methode)

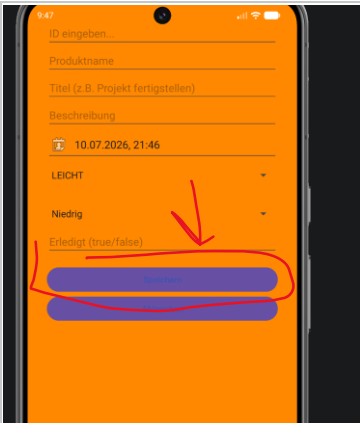
Testprotokoll der User Stories

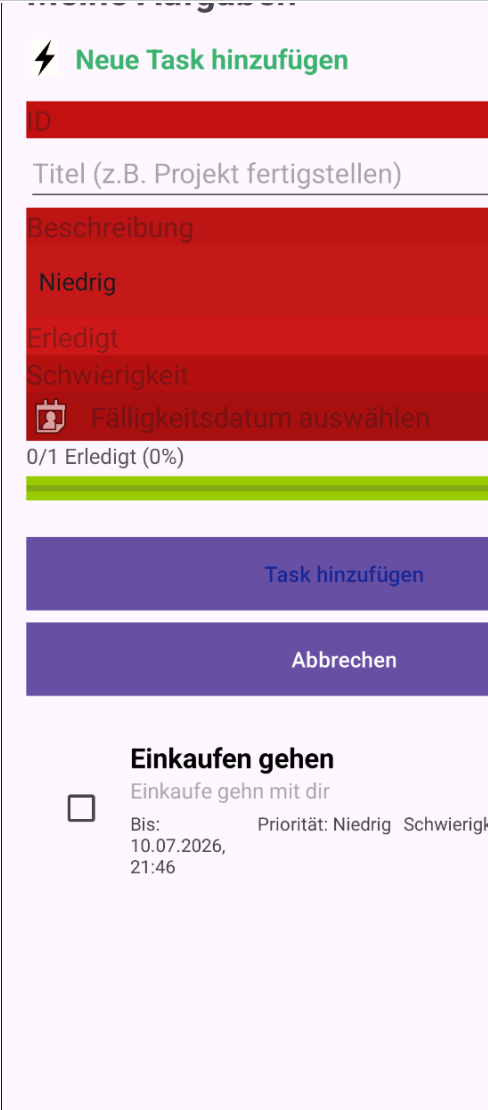
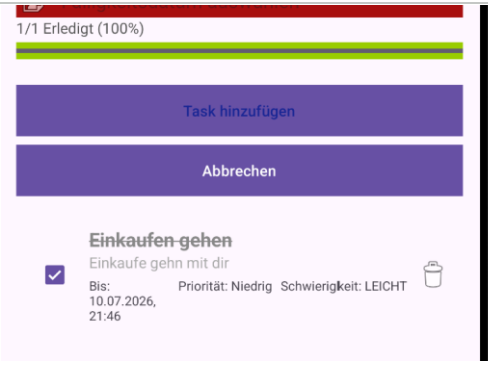
Testprotokoll: Aufgabenplaner Native App

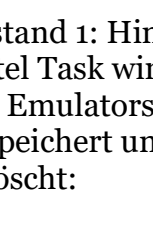
- **System / App:** Aufgabenplaner Native App (Android / Room DB / SQLite) mit VDI und Emulator-Modus
- **Testumgebung:** Android Studio Emulator (VDI) mit Android API 37 für Emulator
- **Tester:** Elias Kaiser
- **Datum:** 07.07.2026

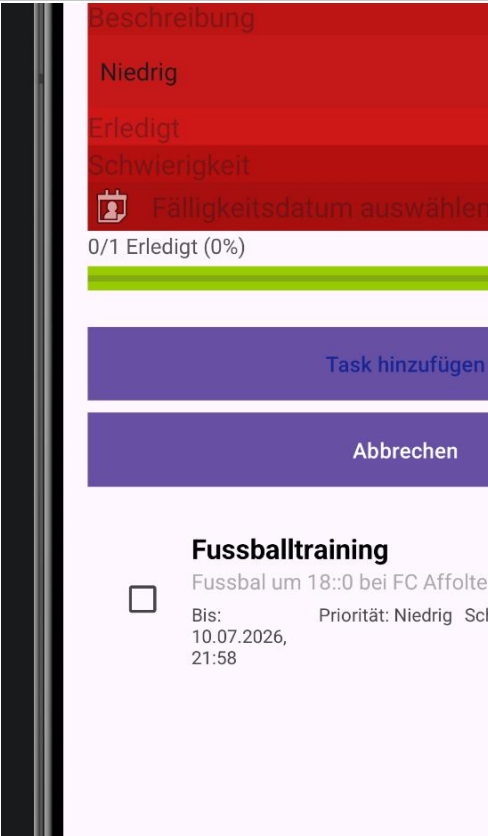
Übersichtstabelle der Testfälle

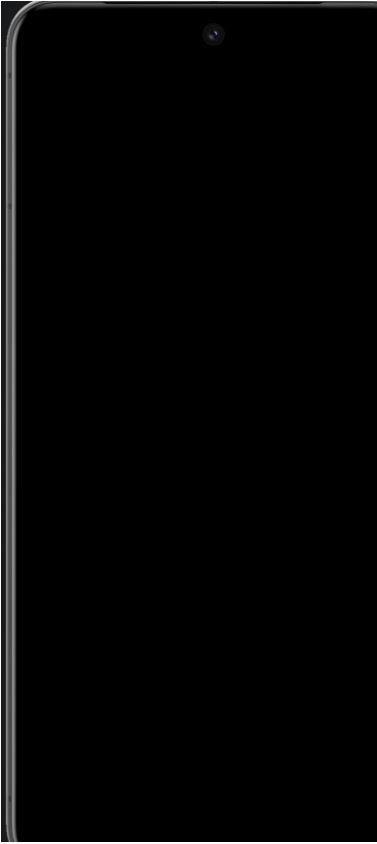
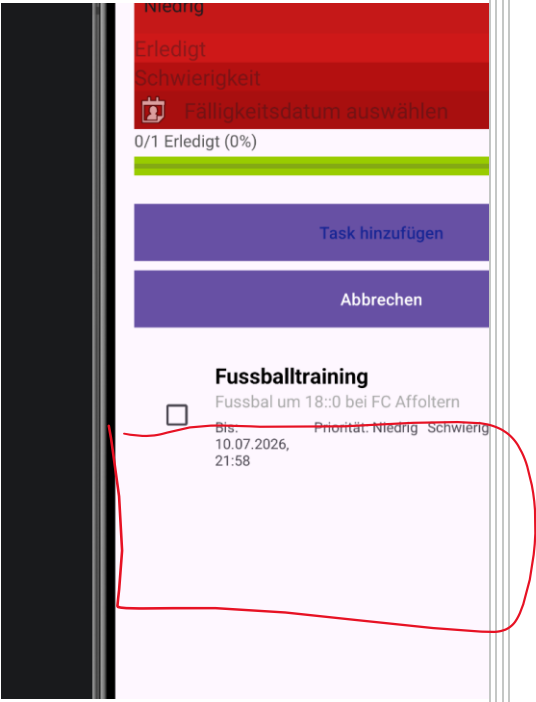
Test-ID	User Story	Testfall-Beschreibung	Tatsächliches Ergebnis	Erwartetes Ergebnis	Status
TF 01	US 1: Tasks übersichtlich anzeigen	Menüsicht		Leere Liste wird gezeigt; Neue Tasks rendern funktioniert korrekt	PASSED
TF 02	US 2: Neue auf Liste Task erfassen	Button Task hinzufügen und Task einkaufen hinzufügen	 Zweite Activity AddTaskItem erscheint:	Formular schliesst sich; Inputfelder (Titel, Description, Difficulty) werden verworfen und beim nächsten Öffnen geleert angezeigt wenn auf Button «Abbrechen» geklickt wird	PASSED

Te st- ID	User Story	Testfall- Beschre ibung	Tatsächlichs Ergebnis	Erwartetes Ergebnis	Stat us
			 Home-Task View: Tasliste angezeigt mit neuem Eintrag Einkaufen gehen:		

Test-ID	User Story	Testfall-Beschreibung	Tatsächliches Ergebnis	Erwartetes Ergebnis	Status
					
TF 03	US 3: Task erledigen und abhaken	Task in der Liste erledigen und abhaken durch Checkbox anklicken		Checkbox-Klick streicht Task sofort durch (UI-Update via updateStrokeThrough). Erneuter Klick setzt Zustand auf «Offen»	PASSED
TF 04	US 4: Task löschen	Task aus der Liste über		Klick auf Mülleimer entfernt Task	PASSED

Test-ID	User Story	Testfall-Beschreibung	Tatsächliches Ergebnis	Erwartetes Ergebnis	Status
	von Taskliste per Deletebutton oder Mülleimer	Mülleimer-Icon löschen oder per Button «Löschen» Aus Taskliste	Zustand 1: Vor dem Löschen der Task  Zustand 2: Task verschwindet und Progress Bar wird automatisch zurückgesetzt: 	sofort aus ListView und löscht ihn unwiderruflich aus der Room-Datenbank sobald auf «Abbrechen» geklickt wird erschwundet die Task	
TF 05	US 5: Datenpersistenz der Task überprüfen	Überprüfen ob Task nach App-Neustart persistent in Room/SQLite-DB prüfen ob Tasks noch existieren auf Listview	Zustand 1: Hinzugefügte Task bzw. erstellte Task wird nach Neustart des Emulators bleibt persistent gespeichert und wird nicht gelöscht: 	Nach einem App-Kill/Neustart im Emulator sind alle vorhandenen Tasks inklusive ihres exakten Zustands (offen/durchgestrichen) sichtbar.	PASSED

Test-ID	User Story	Testfall-Beschreibung	Tatsächliches Ergebnis	Erwartetes Ergebnis	Status
			 Zustand nach Neustart de Emualtors: Power Off und wieder An um Datenbanpersistenz der		

Test-ID	User Story	Testfall-Beschreibung	Tatsächliches Ergebnis	Erwartetes Ergebnis	Status
			Room-Database zu testen:  Hinzugefügt Task bleibt erhalten und verschwindet nicht von Liste! 		

Detaillierte Spezifikation der Testfälle

TF 01: Alle Tasks übersichtlich in Liste anzeigen (US 1)

- **Akzeptanzkriterien:**
 - Liste wird über ListView und einen massgeschneiderten TaskAdapter gerendert.
 - Wenn keine Daten vorhanden sind, zeigt der Emulator eine leere Liste.
 - Nach dem Hinzufügen werden Aufgaben strukturiert mit Beschreibung, Name und Schwierigkeit angezeigt.
 - Validierung schlägt fehl mit Alert-Meldung falls Pflichtfelder wie Title, Description, priority beim Hinzufügen fehlen.
- **Erwartetes Ergebnis:** Die MainActivity startet fehlerfrei. Daten werden sauber via TaskAdapter gebunden, gefiltert und dargestellt.
- **Tatsächliches Ergebnis:** Inputfelder mit Description erscheint im Home-Screen erfolgreich

TF 02: Leere Inputfelder bei Abbrechen-Button (US 2 & Testfall 3)

- **Akzeptanzkriterien:**
 - Klick auf «+Add task» öffnet das Formular (AddTaskActivity / task_add.xml).
 - Eingaben in Titel, Beschreibung und Schwierigkeit können jederzeit über den «Abbrechen»-Button abgebrochen werden.
 - Beim Abbrechen werden keine Daten in der Liste oder DB gespeichert.
 - Die Inputfelder werden geleert/zurückgesetzt, sodass beim nächsten Öffnen keine alten Fragmente vorhanden sind.
- **Erwartetes Ergebnis:** Das Formular schliesst sich ohne Datenänderung. Die Felder sind beim nächsten Aufruf komplett leer.
- **Tatsächliches Ergebnis:** Hinzugefügte Task Einkaufen wird nicht validiert und das Formular zurückgesetzt auf leere Felder

TF 03: Task erledigen und in Taskliste abhaken (US 3 & Testfall 4)

- **Akzeptanzkriterien:**
 - Jedes Listenelement besitzt eine Checkbox mit einem OnCheckedChangeListener.
 - Klick auf die Checkbox/den Task triggert das Interface OnTaskClickListener (onTaskCheckChanged).
 - Die Methode updateStrokeThrough aktualisiert das UI sofort und streicht den Text durch.
 - Erneutes Klicken hebt die Durchstreichung auf und setzt den Task zurück auf «offen».
 - Der Status wird asynchron über das Room-DAO in der SQLite-Datenbank aktualisiert.
- **Erwartetes Ergebnis:** UI aktualisiert sich verzögerungsfrei (visuelle Durchstreichung). Der Status wechselt zuverlässig zwischen offen und erledigt.
- **Tatsächliches Ergebnis:** Task nach Anklicken der Task wird durchgestrichen und wird abgehakt

TF 04: Neue Task aus Liste löschen (US 4)

- **Akzeptanzkriterien:**
 - Neben jedem Task existiert ein Mülleimer-Icon mit einem SetOnClickListener.
 - Klick ruft onTaskDeleted auf dem Interface auf.
 - Der Task wird ohne Umwege aus der ListView entfernt.
 - Die Daten werden dauerhaft über das Room-DAO-Objekt aus der Datenbank gelöscht.
 - *Zusatzregel:* Exception-Handling mit Alert-Meldung greift, falls versucht wird, mehr als 10 Tasks gleichzeitig in der Liste zu halten (sofern implementiert).
- **Erwartetes Ergebnis:** Task verschwindet sofort aus dem UI und wird aus der DB gelöscht.
- **Tatsächliches Ergebnis:** Mülleimer-Iconfunktioniert und Task wird durch Löschen von Liste dauerhaft gelöscht

TF 05: Aufgabe wird nach Neustart sichtbar / Persistent in DB (US 5 & Testfall 2)

- **Akzeptanzkriterien:**
 - Alle Attribute (Beschreibung, Priorität/Schwierigkeit, Taskname) stimmen im DAO mit dem Task-Objekt überein.
 - Wird die App via Kill-Prozess im Emulator komplett geschlossen und neu gestartet, wird der exakte Zustand (offen oder durchgestrichen) aller nicht-gelöschten Tasks geladen.
 - Statusänderungen (Erledigt aus US 3) oder Löschungen (aus US 4) bleiben nach dem Neustart konsistent erhalten.
- **Erwartetes Ergebnis:** Nach dem vollständigen Schliessen und Neustarten der App im Emulator rendert die MainActivity alle Tasks fehlerfrei anhand der SQLite/Room-Datenbank. Keine Daten gehen verloren.
- **Tatsächliches Ergebnis:** __ **Neue Task Arbeiten wird erfolgreich naach Neustart des Emulators(Power-off und On) bleibt persitent gespeichert und verschwindet nicht von Home-Screen aus Liste → _____**